Assignment 2 (Weeks 3 and 4)

Part I: Short Answer / Reasoning Questions

Q1. NULL in Comparisons and Aggregates

The query `SELECT COUNT(*) FROM Student WHERE phone_number = NULL` always returns 0 because `NULL` represents an unknown or missing value, not a comparable value. In SQL's three-valued logic (TRUE / FALSE / UNKNOWN), any comparison involving `NULL` — including `phone_number = NULL` — evaluates to UNKNOWN rather than TRUE, even for rows where `phone_number` actually is `NULL`. The `WHERE` clause only keeps rows where the condition evaluates to TRUE, so every row is discarded and the count is 0.

The corrected query is:

```
SELECT COUNT(*)
FROM Student
WHERE phone_number IS NULL;
```

In general, `NULL` is excluded from most aggregate computations (`SUM`, `AVG`, `MIN`, `MAX`, and the single-column form of `COUNT`) because these functions are defined to operate only over known, comparable values; including an "unknown" value in a sum or average would make the result itself meaningless. Similarly, standard comparison operators (`=`, `<`, `>`, etc.) always return UNKNOWN when either operand is `NULL`, since SQL cannot determine whether an unknown value satisfies the comparison, which is why explicit `IS NULL` / `IS NOT NULL` checks exist.

Q2. Is DISTINCT Redundant with GROUP BY?

The student is correct — `DISTINCT` is redundant here. The query is:

```
SELECT DISTINCT department, COUNT(*)
FROM Employee
GROUP BY department;
```

`GROUP BY department` already collapses the entire `Employee` table into exactly one output row per distinct department value, with `COUNT(*)` computed once per group. Since `DISTINCT` is applied to the final result set of `(department, count)` pairs, and each department can only appear once in that result (because it is the grouping key), there are no duplicate rows left for `DISTINCT` to remove. `DISTINCT` would only do meaningful work if the `SELECT` list could contain repeated combinations after grouping, which cannot happen when the grouping column itself is present in the output.

Q3. INNER JOIN vs. LEFT JOIN Row-Count Difference

A difference in row count between an `INNER JOIN` and a `LEFT JOIN` (`Customer` as the left table) on `Customer` and `Order` tells you that at least one customer row has no matching row in the `Order` table — i.e., there exist customers who have never placed an order (or whose orders reference a

customer_id not matched by the join condition). INNER JOIN discards any Customer row that lacks a matching Order row, while LEFT JOIN keeps every Customer row regardless, filling in NULLs for the Order columns when no match exists.

The LEFT JOIN will return more rows than the INNER JOIN precisely in the scenario where one or more customers exist in the Customer table with zero corresponding rows in the Order table (new customers who registered but never ordered, or customers whose account is inactive). Each such customer contributes exactly one extra row (with NULL order columns) under LEFT JOIN that simply would not appear under INNER JOIN.

Q4. Why the Ungrouped-Column Query Is Invalid

```
SELECT department, employee_name, AVG(salary)
FROM Employee
GROUP BY department;
```

This query is invalid in standard SQL because employee_name appears in the SELECT list but is neither part of the GROUP BY clause nor wrapped inside an aggregate function. GROUP BY department collapses all employees sharing a department into a single group/row, so unless a department happens to contain only one employee, there is no single, well-defined employee_name to display for that group — SQL would have to arbitrarily pick one of several possible values, which violates the requirement that every non-aggregated SELECT column be functionally dependent on the GROUP BY key. If SQL "allowed" this query to run as written, employee_name would have to resolve to some unspecified, essentially non-deterministic value (an arbitrary representative employee from the group), which would be misleading and non-reproducible — hence standard SQL rejects it outright rather than silently guessing.

Corrected (aggregate query, one row per department):

```
SELECT department, AVG(salary)
FROM Employee
GROUP BY department;
```

Or, if per-employee detail is genuinely required, group by employee_name as well (which changes the granularity to one group per employee):

```
SELECT department, employee_name, salary AS avg_salary
FROM Employee
GROUP BY department, employee_name, salary;
```

Q5. WHERE vs. HAVING and Aggregate Filtering

```
SELECT department, AVG(salary)
FROM Employee
WHERE AVG(salary) > 80000
GROUP BY department;
```

The error is using an aggregate function (AVG) directly inside a WHERE clause. In SQL's logical query-processing order, FROM is evaluated first, then WHERE, then GROUP BY, then the aggregate

functions are computed, and only after that is `HAVING` applied, followed finally by `SELECT`. Because `WHERE` executes before grouping and aggregation take place, `AVG(salary)` simply does not exist yet at the point `WHERE` is evaluated — there is no per-department average to compare against for a given row. Filtering on the result of an aggregate must therefore happen in the `HAVING` clause, which runs after groups and their aggregates have been formed.

Corrected query:

```
SELECT department, AVG(salary)
FROM Employee
GROUP BY department
HAVING AVG(salary) > 80000;
```

Q6. Correlated vs. Non-Correlated Subqueries

(a) Products priced above the overall average product price

Non-correlated subquery. The "overall average product price" is a single, fixed value computed once across the entire `Product` table; it does not depend on any value from the outer query's current row, so it can be evaluated independently and reused for every comparison.

```
SELECT product_id, price
FROM Product
WHERE price > (SELECT AVG(price) FROM Product);
```

(b) Employees earning more than their own department's average salary

Correlated subquery. Here the threshold ("average salary of their own department") is different for every employee depending on which department that employee belongs to, so the inner query must reference the outer row's department for each evaluation.

```
SELECT e.employee_id, e.salary, e.department
FROM Employee e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM Employee e2
    WHERE e2.department = e.department
);
```

Performance implication: a non-correlated subquery is logically evaluated once and its result reused for every row of the outer query, so its cost is essentially independent of the outer table's size. A correlated subquery, by contrast, is conceptually re-evaluated once per outer row (unless the query optimizer is able to rewrite it, e.g., as a join with a pre-aggregated derived table), so its cost tends to grow with the number of outer rows — on a large outer table this can be significantly slower than an equivalent non-correlated or join-based formulation.

Part II: Long Answer — Online Food Delivery Schema

Schema reused from Assignment 1: `Customer(customer_id, name, email, phone)`, `Restaurant(restaurant_id, name, cuisine_type, rating)`, `MenuItem(item_id, restaurant_id, item_name, price)`, `Order(order_id,`

customer_id, restaurant_id, order_date, status), OrderItem(order_id, item_id, quantity).

(a) Customers Who Have Never Placed an Order

```
SELECT c.customer_id, c.name
FROM Customer c
LEFT JOIN "Order" o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Equivalent NOT EXISTS form (also correct, and avoids any ambiguity around NULLs in the correlated column):

```
SELECT c.customer_id, c.name
FROM Customer c
WHERE NOT EXISTS (
    SELECT 1 FROM "Order" o WHERE o.customer_id = c.customer_id
);
```

Justification: Capturing "never placed an order" requires keeping every Customer row regardless of whether a match exists in Order, then filtering for the absence of a match. A plain INNER JOIN would only ever return customers who *do* have matching orders, so it cannot express "never" at all. LEFT JOIN ...WHERE right_key IS NULL (or equivalently NOT EXISTS) is the standard pattern for anti-joins. NOT EXISTS is generally preferred over NOT IN here because NOT IN silently misbehaves (returns no rows) if the subquery's column can contain NULLs, whereas NOT EXISTS and LEFT JOIN/IS NULL are unaffected by that.

(b) Average Order Value per Restaurant, Restaurants with More than 5 Orders

```
SELECT t.restaurant_id, AVG(t.order_value) AS avg_order_value
FROM (
    SELECT o.restaurant_id, o.order_id,
           SUM(mi.price * oi.quantity) AS order_value
    FROM "Order" o
    JOIN OrderItem oi ON o.order_id = oi.order_id
    JOIN MenuItem mi ON oi.item_id = mi.item_id
    GROUP BY o.restaurant_id, o.order_id
) t
GROUP BY t.restaurant_id
HAVING COUNT(*) > 5;
```

Justification: An order's value is itself an aggregate (sum of price × quantity across its items), so it must be computed first at the order grain before it can be averaged at the restaurant grain — SQL does not allow nesting two aggregate functions like AVG(SUM(...)) in a single GROUP BY level, hence the inner derived table. HAVING (not WHERE) is used to filter restaurants by "more than 5 orders" because that condition depends on COUNT(*), an aggregate that is only available after grouping.

(c) Restaurants Whose Average Order Value Exceeds the Platform-Wide Average

```
SELECT r.restaurant_id, r.avg_order_value
FROM (
    SELECT t.restaurant_id, AVG(t.order_value) AS avg_order_value
    FROM (
        SELECT o.restaurant_id, o.order_id,
               SUM(mi.price * oi.quantity) AS order_value
        FROM "Order" o
        JOIN OrderItem oi ON o.order_id = oi.order_id
        JOIN MenuItem mi ON oi.item_id = mi.item_id
        GROUP BY o.restaurant_id, o.order_id
    ) t
    GROUP BY t.restaurant_id
) r
WHERE r.avg_order_value > (
    SELECT AVG(all_orders.order_value)
    FROM (
        SELECT o.order_id,
               SUM(mi.price * oi.quantity) AS order_value
        FROM "Order" o
        JOIN OrderItem oi ON o.order_id = oi.order_id
        JOIN MenuItem mi ON oi.item_id = mi.item_id
        GROUP BY o.order_id
    ) all_orders
);
```

Justification: This does not need to be a correlated subquery. The "platform-wide average order value" is one single number computed across all orders on the platform — it does not depend on which restaurant row from the outer query is currently being evaluated. So the inner average is computed once and reused as a constant threshold for every restaurant, which is both simpler to read and cheaper to run than re-deriving the platform average once per restaurant, as a correlated version would needlessly do.

(d) Most-Ordered Menu Item (by Total Quantity) per Restaurant

```
SELECT mi.restaurant_id, mi.item_id, mi.item_name, totals.total_qty
FROM (
    SELECT mi2.restaurant_id, oi.item_id, SUM(oi.quantity) AS total_qty
    FROM OrderItem oi
    JOIN "Order" o ON oi.order_id = o.order_id
    JOIN MenuItem mi2 ON oi.item_id = mi2.item_id
    GROUP BY mi2.restaurant_id, oi.item_id
) totals
JOIN MenuItem mi ON mi.item_id = totals.item_id
WHERE totals.total_qty = (
    SELECT MAX(t2.total_qty)
    FROM (
        SELECT mi3.restaurant_id, oi2.item_id, SUM(oi2.quantity) AS total_qty
        FROM OrderItem oi2
```

```
        JOIN "Order" o2 ON oi2.order_id = o2.order_id
        JOIN MenuItem mi3 ON oi2.item_id = mi3.item_id
        GROUP BY mi3.restaurant_id, oi2.item_id
    ) t2
    WHERE t2.restaurant_id = totals.restaurant_id
);
```

Justification: Total quantity per item must first be aggregated per `(restaurant, item)` pair; the outer query then keeps only the row(s) whose `total_qty` equals the maximum for that same restaurant, found via a correlated subquery keyed on `restaurant_id`.

Limitation of `GROUP BY` + `MAX` for "top-N per group": this approach naturally returns all items that tie for the maximum quantity in a restaurant, which may be desirable or may silently produce more than one "top item" per restaurant when a tie exists. It also does not generalize cleanly — extending it to a genuine "top-3 items per restaurant" would require increasingly awkward correlated subqueries or self-joins. A window function such as `RANK()` or `ROW_NUMBER() PARTITION BY restaurant_id ORDER BY total_qty DESC` handles both ties and arbitrary top-N cases far more cleanly, but is not used here since it falls outside the `GROUP BY`/`HAVING`/subquery constructs this assignment targets.

Part III: Normalization

```
Enrollment(student_id, student_name, course_id, course_name,
          instructor_id, instructor_name, instructor_office,
          grade, enrollment_date)
```

Q1. Functional Dependencies Implied by the Scenario

- `student_id` → `student_name`
- `course_id` → `course_name`, `instructor_id`
- `instructor_id` → `instructor_name`, `instructor_office`
- `course_id` → `instructor_office` (transitively, via `instructor_id`)
- `(student_id, course_id)` → `grade`, `enrollment_date`

Q2. Is the Table in 1NF?

Yes. Assuming every column holds a single atomic value per row (no repeating groups, no multi-valued fields such as a list of grades in one cell), the table satisfies 1NF. Each row represents one student–course enrollment with a fixed set of single-valued attributes, so there is no violation of atomicity at this level.

Q3. Partial Dependency Violating 2NF

The candidate key is the composite `(student_id, course_id)`. `student_id` → `student_name` and `course_id` → `course_name, instructor_id, instructor_name, instructor_office` are partial dependencies, because each depends on only part of the composite key (`student_id` alone, or `course_id` alone) rather than on the whole key together. This violates 2NF.

Anomaly example (update): if a student's name is corrected (e.g., a spelling fix), the change must be applied to every row in which that student appears — one row per course they're enrolled in. Missing even one row leaves the table in an inconsistent state where the same `student_id` maps to two different names. Similar risks apply on the course/instructor side: an insertion anomaly arises because a new course with no students yet enrolled cannot be recorded at all (the key requires a `student_id`), and a deletion anomaly arises because removing the only enrollment row for a course erases all record of that course's name and instructor.

Q4. Transitive Dependency Violating 3NF

`course_id` → `instructor_id`, and `instructor_id` → `instructor_name, instructor_office`. This chain means `instructor_name` and `instructor_office` depend on `course_id` only transitively, through the non-key attribute `instructor_id`, rather than depending directly on the key. This is a transitive dependency and violates 3NF.

Anomaly example: if an instructor moves to a new office, every row for every course that instructor teaches must be updated with the new office — an update anomaly if any row is missed. An insertion anomaly occurs if the school wants to record a newly hired instructor's office before they are assigned to teach any course, since there is no row to attach that information to without a `course_id`. A deletion anomaly occurs if the last course taught by an instructor is removed from the table (e.g., the course is discontinued), which would also erase the instructor's name and office information entirely.

Q5. Decomposition into 3NF

```
Student(student_id, student_name)
Instructor(instructor_id, instructor_name, instructor_office)
Course(course_id, course_name, instructor_id)
Enrollment(student_id, course_id, grade, enrollment_date)
```

Primary keys: `Student.student_id`; `Instructor.instructor_id`; `Course.course_id`; `Enrollment.(student_id course_id)` as a composite key. `Course.instructor_id` and `Enrollment.student_id` / `Enrollment.course_id` are foreign keys referencing `Instructor` and `Student`/`Course` respectively.

- **Student** — separated out because `student_id` → `student_name` held only on part of the original composite key.
- **Instructor** — separated out because `instructor_id` → `instructor_name, instructor_office` was a transitive dependency (via `course_id` → `instructor_id`).

- **Course** — separated out because `course_id` → `course_name, instructor_id` held only on part of the original composite key; it retains `instructor_id` as a foreign key so a course's instructor can still be looked up.
- **Enrollment** — what remains after removing the partial and transitive dependencies: `grade` and `enrollment_date` are the only attributes that genuinely depend on the full composite key `(student_id, course_id)`.

Part IV: Design Justification

In Part II(a), "customers who have never placed an order" could have been written either as a `LEFT JOIN` followed by `WHERE order_id IS NULL`, or as a `NOT EXISTS` correlated subquery; both are semantically equivalent anti-join patterns. I favored the `LEFT JOIN` form for its readability — it makes the "keep everything from `Customer`, mark what's missing" intent visually explicit — but I noted `NOT EXISTS` as the safer default in general, since a naive `NOT IN` alternative would silently return zero rows if the `Order.customer_id` column could ever contain a `NULL`, a correctness trap that `NOT EXISTS` and `LEFT JOIN/IS NULL` both avoid. For most query optimizers the performance of the two forms is comparable, so correctness-under-`NULL`s and readability were the deciding factors rather than raw speed.

In Part III, normalizing `Enrollment` into four tables removes update/insertion/deletion anomalies, but makes reads harder: retrieving a single "roster view" with student name, course name, and instructor details now requires joining four tables instead of reading one row. A real system handling heavy read traffic (e.g., a dashboard queried far more often than it is updated) might intentionally denormalize — keeping a wide, precomputed reporting table or materialized view — trading some redundancy and anomaly risk for faster, simpler reads.